

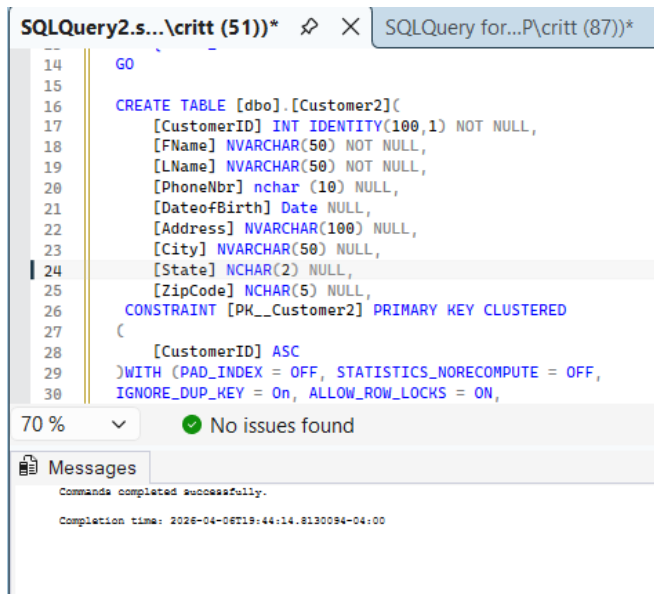
Ethan Krul

ISIN 325

4/6/2026

SQL3

- 1) SQL to create a Customer2 table using appropriate data types that has the following columns: CustomerID, First Name, Last Name, Date of Birth, Address, City, State, Zip Code. CustomerID must be defined as an identity field that starts with 100.
- 2) SQL to define the primary key in the CustomerID column. (SQL must not be generated)



```
--
14 GO
15
16 CREATE TABLE [dbo].[Customer2](
17     [CustomerID] INT IDENTITY(100,1) NOT NULL,
18     [FName] NVARCHAR(50) NOT NULL,
19     [LName] NVARCHAR(50) NOT NULL,
20     [PhoneNbr] nchar(10) NULL,
21     [DateofBirth] Date NULL,
22     [Address] NVARCHAR(100) NULL,
23     [City] NVARCHAR(50) NULL,
24     [State] NCHAR(2) NULL,
25     [ZipCode] NCHAR(5) NULL,
26     CONSTRAINT [PK_Customer2] PRIMARY KEY CLUSTERED
27     (
28         [CustomerID] ASC
29     )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
30     IGNORE_DUP_KEY = On, ALLOW_ROW_LOCKS = ON,
```

70 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-04-06T19:44:14.8130094-04:00

- 3) SQL to define an index using the primary key and a second index using Last Name and First Name. (SQL must not be generated)

```
SQLQuery2.s... \critt (51))* X SQLQuery for...P\critt (87))*
1 --3) SQL to define an index using the p
2 CREATE INDEX NameIDX --creates index
3 ON Customer2 (LName, FName);
4
```

100 % 3 0 ↑ ↓

Messages

Commands completed successfully.

Completion time: 2026-04-06T19:45:40.4932733-04:00

4) SQL to insert fifty rows of unique data into the Customer2 table.

```
Ethanslaptop...bo.Customer2 SQLQuery2.s... \critt (51))* X SQLQuery for...P\critt (87))*
1 --4) SQL to insert fifty rows of unique data into the Customer2 table.
2 INSERT INTO [dbo].[Customer2] (FName, LName, PhoneNbr, DateOfBirth, Address, City, State, ZipCode) VALUES
3 ('John', 'Smith', '6165550001', '1985-03-12', '123 Oak St', 'Grand Rapids', 'MI', '49503'),
4 ('Emma', 'Johnson', '6165550002', '1990-07-25', '456 Pine Ave', 'Rockford', 'MI', '49341'),
5 ('Liam', 'Williams', '6165550003', '1978-11-02', '789 Maple Rd', 'Lansing', 'MI', '48910'),
6 ('Olivia', 'Brown', '6165550004', '2000-01-15', '321 Birch Ln', 'Detroit', 'MI', '48201'),
7 ('Noah', 'Jones', '6165550005', '1995-06-30', '654 Cedar St', 'Holland', 'MI', '49423'),
8 ('Ava', 'Garcia', '6165550006', '1988-09-09', '987 Spruce Dr', 'Kalamazoo', 'MI', '49007'),
9 ('Elijah', 'Miller', '6165550007', '1975-04-18', '159 Walnut St', 'Ann Arbor', 'MI', '48103'),
10 ('Sophia', 'Davis', '6165550008', '1992-12-05', '753 Chestnut Ave', 'Flint', 'MI', '48502'),
11 ('Lucas', 'Rodriguez', '6165550009', '1983-08-21', '852 Poplar Rd', 'Muskegon', 'MI', '49441'),
12 ('Mia', 'Martinez', '6165550010', '1999-02-10', '951 Ash Ct', 'Wyoming', 'MI', '49509'),
```

100 % 9 0 ↑ ↓

Messages

(50 rows affected)

Completion time: 2026-04-06T19:49:09.2144373-04:00

5) SQL to truncate and drop the Customer2 table.

```
SQLQuery4.s... \critt (51))* X SQLQuery for...P\critt (87))*
1 --5) SQL to truncate and drop the Customer2 table.
2 TRUNCATE TABLE [dbo].[Customer2];
3
4 DROP TABLE [dbo].[Customer2];
5
```

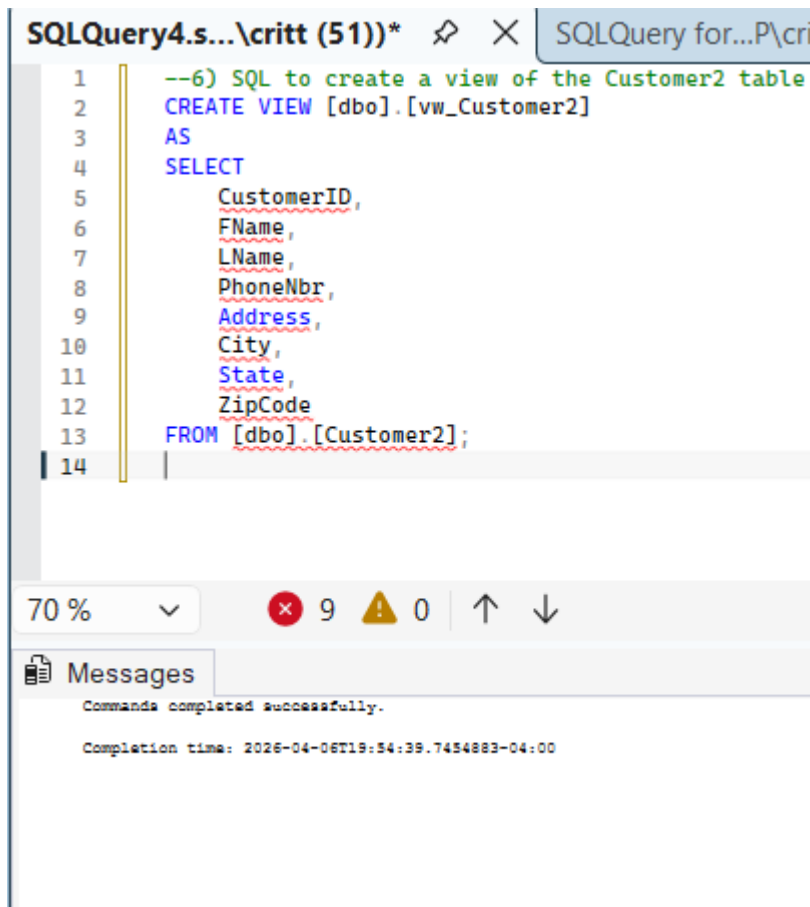
100 % 1 0 ↑ ↓

Messages

Commands completed successfully.

Completion time: 2026-04-06T19:52:37.9619763-04:00

6) SQL to create a view of the Customer2 table using all of the columns in the table except Date of Birth.



The screenshot shows a SQL Server Enterprise Manager window with a query editor. The query is as follows:

```
1  --6) SQL to create a view of the Customer2 table
2  CREATE VIEW [dbo].[vw_Customer2]
3  AS
4  SELECT
5      CustomerID,
6      FName,
7      LName,
8      PhoneNbr,
9      Address,
10     City,
11     State,
12     ZipCode
13  FROM [dbo].[Customer2];
14
```

Below the query editor, the status bar shows 70% zoom, 9 errors, and 0 warnings. The Messages pane at the bottom indicates that the command was completed successfully and provides the completion time: 2026-04-06T19:54:39.7454883-04:00.

7) SQL that uses the view to display all Customers in the Customer2 table sorted by First Name in descending order.

SQLQuery4.s... \critt (51))* SQLQuery for...P\critt (87))*

```

1  --7) SQL that uses the view to display all Customers in the Customer2 table sorted by Fir
2  SELECT *
3  FROM [dbo].[vw_Customer2]
4  ORDER BY FName DESC;

```

70 % 2 0 ↑ ↓

Results Messages

	CustomerID	FName	LName	PhoneNbr	Address	City	State	ZipCode
1	139	Zoey	Nelson	6165550040	676 Berry St	Eastpointe	MI	48021
2	131	Victoria	Young	6165550032	707 Pineapple St	Shelby Twp	MI	48315
3	107	Sophia	Davis	6165550008	753 Chestnut Ave	Flint	MI	48502
4	128	Sebastian	Lewis	6165550029	404 Melon St	Warren	MI	48089
5	125	Scarlett	Sanchez	6165550026	101 Berry Ln	Southfield	MI	48075
6	138	Samuel	Adams	6165550039	565 Peach Dr	Roseville	MI	48066
7	147	Riley	Evans	6165550048	555 Orange Rd	Taylor	MI	48180
8	103	Olivia	Brown	6165550004	321 Birch Ln	Detroit	MI	48201
9	141	Nora	Campbell	6165550042	898 Plum Rd	Grosse Pointe	MI	48230
10	104	Noah	Jones	6165550005	654 Cedar St	Holland	MI	49423
...	...	...	...	...	...	...	...	...

Now use the Meijer database ->

8) One sql routine with at least 3 If statements and confirmation that they work.

SQLQuery4.s... \critt (51))* SQLQuery for...P\critt (87))*

```

1  /*Now use the Meijer database -> */
2
3  --8) One sql routine with at least 3 If statements and confirmation that they work.
4  Declare @Counter int = 3
5  if @Counter < 2
6      Print 'Less than 2'
7  If @Counter = 2
8      print 'Equal 2'
9  If @Counter > 2
10     Print 'Greater than 2' + CAST(@Counter As Varchar(10))
11

```

70 % No issues found

Messages

Greater than 23

Completion time: 2026-04-06T19:56:28.3173829-04:00

9) One sql routine with If, Else-If and confirmation that they work.

```
SQLQuery4.s...\critt (51))*  X  SQLQuery for...P\critt (87))*
1  --9) One sql routine with If, Else-If and confirmation that they work.
2  Declare @Counter int = 1
3  IF @Counter < 2
4      Print 'Less than 2'
5  Else if @Counter = 2
6      print 'Equal 2'
7  Else Print ' Greater than 2- value is: ' + CAST(@Counter as Varchar(10))

100 %  No issues found
Messages
Less than 2
Completion time: 2026-04-06T19:57:41.1422029-04:00
```

10) One Case statement with at least 3 When options and confirmation that they work.

```
SQLQuery4.s...\critt (51))*  X  SQLQuery for...P\critt (87))*
1  --10) One Case statement with at least 3 When options and confi:
2  Declare @Counter Int = 3
3  Print
4      Case
5          When @Counter < 2 THEN 'Less than 2'
6          When @Counter = 2 THEN 'Equal 2'
7          Else CONCAT('Greater than 2 - value is: ', @Counter)
8      END;
```

```
100 %  No issues found
Messages
Greater than 2 - value is: 3
Completion time: 2026-04-06T19:58:08.6050492-04:00
```

11) Alter table sql to add an integer column to an existing table that allows nulls.

```
SQLQuery4.s...\critt (51))*  X  SQLQuery for...P\critt (87))*
1  --11) Alter table sql to add an integer column to an existing table that allows nulls.
2  ALTER TABLE Customer2
3  ADD Age INT NULL;
4
```

```
100 %  No issues found
Messages
Commands completed successfully.
Completion time: 2026-04-06T20:00:12.9119517-04:00
```

12) SQL update routine with error checking and error logging. Confirmation statement that generates logged error.

SQLQuery4.s... \critt (51))* × SQLQuery for...P\critt (87))*

```

1  --12) SQL update routine with error checking and error logging. Confirmation statement that generates logged error.
2  DECLARE @ErrorHold INT;
3  INSERT INTO Customer2 (CustomerID, FName) -- This will generate an error because CustomerID is an identity column
4  VALUES (51, 'John');
5  SET @ErrorHold = @@ERROR; -- Capture the error
6  PRINT CONCAT('@@error is: ', @ErrorHold); -- Print confirmation
7  IF @ErrorHold != 0 -- Log the error if it exists
8  BEGIN
9      INSERT INTO ErrorAudit (ErrorNbr, ErrorTime)
10     VALUES (@ErrorHold, GETDATE());
11 END

```

100 % 6 0 ↑ ↓

Messages

@@error is: 544
(1 row affected)
Completion time: 2026-04-06T20:03:11.2288660-04:00

13) SQL statement that defines and uses variables.

SQLQuery4.s... \critt (51))* × SQLQuery for...P\critt (87))*

```

1  --13) SQL statement that defines and uses variables.
2  DECLARE @FirstName NVARCHAR(50); -- Define variables and use them
3  DECLARE @LastName NVARCHAR(50);
4  DECLARE @City NVARCHAR(50);
5  SET @FirstName = 'Alice'; -- Assign values to the variables
6  SET @LastName = 'Walker';
7  SET @City = 'Grand Rapids';
8  PRINT CONCAT(@FirstName, ' ', @LastName, ' lives in ', @City); -- Use the variables in a SELECT or PRINT statement

```

100 % No issues found Ln: 8, Ch: 2, Cc

Messages

Alice Walker lives in Grand Rapids
Completion time: 2026-04-06T20:05:32.0555598-04:00

14) Store procedure that accepts a variable and has error checking and logging as part of the execute stored procedure statement.

SQLQuery4.s... \critt (51))* × SQLQuery for...P\critt (87))*

```

1  --14) Store procedure that accepts a variable and has error checking and logging as part of the execute stored procedure statement.
2  CREATE PROCEDURE InsertCustomer -- Create the procedure
3  @FName NVARCHAR(50), -- Input parameter: First Name
4  @LName NVARCHAR(50) -- Input parameter: Last Name
5  AS
6  BEGIN
7      DECLARE @ErrorHold INT; -- Declare a variable to hold any error codes
8      INSERT INTO Customer2 (FName, LName) -- Attempt to insert the new customer into Customer2
9      VALUES (@FName, @LName);
10     SET @ErrorHold = @@ERROR; -- Capture any error that occurred during the insert
11     IF @ErrorHold != 0 -- If there was an error, log it into the ErrorAudit table
12     BEGIN
13         INSERT INTO ErrorAudit (ErrorNbr, ErrorTime)
14         VALUES (@ErrorHold, GETDATE());
15     END
16 END;
17 EXEC InsertCustomer 'John', 'Doe'; -- Execute the procedure to test it

```

70 % 6 0 ↑ ↓

Messages

Commands completed successfully.
Completion time: 2026-04-06T20:06:06.1062120-04:00

15) SQL insert statement that inserts ten rows of data into a new table named Cars that includes at least 5 columns and the data varies for each record by using variables that are incremented and/or changed within the routine for each record.

```
SQLQuery4.s...\critt (51))* X SQLQuery for...P\critt (87))*
1  --15) SQL insert statement that inserts ten rows of data into a new table na
2  --and the data varies for each record by using variables that are incremente
3  CREATE TABLE Cars ( --Create the Cars table
4      CarID INT IDENTITY(1,1),
5      Make NVARCHAR(50),
6      Model NVARCHAR(50),
7      Year INT,
8      Price DECIMAL(10,2),
9      Mileage INT
10 );
11 DECLARE @Counter INT = 1; -- Declare a counter variable for looping
12 WHILE @Counter <= 10 --Loop to insert 10 rows with varying data
13 BEGIN
14     INSERT INTO Cars (Make, Model, Year, Price, Mileage)
15     VALUES (
16         CONCAT('Make', @Counter),
17         CONCAT('Model', @Counter),
18         2010 + @Counter,
19         20000 + (@Counter * 1000),
20         50000 - (@Counter * 2000)
21     );
22
23     SET @Counter = @Counter + 1; -- Increment the counter
24 END;
25
```

100 % No issues found

Messages

(1 row affected)

(1 row affected)

(1 row affected)

100 % No issues found

16) Create a table named HikingLocations that has the appropriate structure to remove duplicates when loading the table. Demonstrate that it works. Include sql that defines the structure to remove duplicates in your submission. (this is the same as any other table create except you set the ignore_dup_key option to on).

```
SQLQuery4.s...\critt (51))* X SQLQuery for...P\critt (87))*
1  --16) Create a table named HikingLocations that has the appropriate structure to remove duplicates when loading
2  --Demonstrate that it works. Include sql that defines the structure to remove duplicates in your submission.
3  --(this is the same as any other table create except you set the ignore_dup_key option to on).
4  CREATE TABLE HikingLocations ( --creates table
5      LocationID INT IDENTITY(1,1),
6      LocationName NVARCHAR(100),
7      State NVARCHAR(50),
8      CONSTRAINT UQ_Hiking UNIQUE (LocationName, State)
9      WITH (IGNORE_DUP_KEY = ON) -- Ignores duplicate inserts instead of throwing an error
10 );
11
12 INSERT INTO HikingLocations (LocationName, State) -- First insert works normally
13 VALUES ('Trail A', 'MI'),
14 ('Trail A', 'MI'); -- Because of IGNORE_DUP_KEY = ON, this insert is ignored and does NOT throw an error
15
```

100 % No issues found

Messages

Duplicate key was ignored.

(1 row affected)

Completion time: 2026-04-06T20:11:42.6924856-04:00

17) Define a table named HikingLocations2 that has 10 records and 3 of the records are duplicates. Create an SQL routine that identifies the duplicate records by key value. Create a second SQL routine that displays only the duplicate records.

SQLQuery4.s... \critt (51))* X SQLQuery for...P\critt (87))*

```

1  --17) Define a table named HikingLocations2 that has 10 records and 3 of the records are duplicate
2  CREATE TABLE HikingLocations2 ( -- Create table
3      LocationName NVARCHAR(100),
4      State NVARCHAR(50)
5  );
6  INSERT INTO HikingLocations2 VALUES -- Insert 10 records, 3 duplicates
7      ('Trail A','MI'),
8      ('Trail B','MI'),
9      ('Trail C','MI'),
10     ('Trail A','MI'),
11     ('Trail B','MI'),
12     ('Trail D','MI'),
13     ('Trail E','MI'),
14     ('Trail F','MI'),
15     ('Trail C','MI'),
16     ('Trail G','MI');
17  SELECT -- Identify duplicates by key value -- Groups by LocationName and State, counts occurrences
18      LocationName,
19      State,
20      COUNT(*) AS Occurrences
21  FROM HikingLocations2
22  GROUP BY LocationName, State
23  HAVING COUNT(*) > 1;
24  SELECT h -- Display only the duplicate records.
25  FROM HikingLocations2 h
26  INNER JOIN (
27      SELECT LocationName, State
28      FROM HikingLocations2
29      GROUP BY LocationName, State
30      HAVING COUNT(*) > 1
31  ) dup
32  ON h.LocationName = dup.LocationName
33  AND h.State = dup.State;
34  --

```

70 % No issues found

LocationName	State	Occurrences
Trail A	MI	2
Trail B	MI	2
Trail C	MI	2

LocationName	State
Trail A	MI
Trail A	MI
Trail B	MI

18) Create an insert routine for HikingLocations2 that uses variables and performs error checking and error logging.

SQLQuery4.s... \critt (51))* X SQLQuery for...P\critt (87))*

```

1  --18) Create an insert routine for HikingLocations2 that
2  DECLARE @LocName NVARCHAR(100);
3  DECLARE @State NVARCHAR(50);
4  DECLARE @ErrorHold INT;
5
6  SET @LocName = 'Trail X';
7  SET @State = 'MI';
8
9  INSERT INTO HikingLocations2 (LocationName, State)
10     VALUES (@LocName, @State);
11
12  SET @ErrorHold = @@ERROR;
13
14  IF @ErrorHold != 0
15  BEGIN
16      INSERT INTO ErrorAudit (ErrorNbr, ErrorTime)
17      VALUES (@ErrorHold, GETDATE());
18  END

```

100 % 6 0 ↑ ↓

Messages

(1 row affected)

Completion time: 2026-04-06T20:16:52.3236306-04:00